Multi-Agent Execution

Haripreeth Avarur, Bhargav KS, Janani Giridharan, Shreya Nadiger



MICHIGAN ENGINEERING
COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY OF MICHIGAN

PAPER 1 OF 2

Towards a Science of Scaling Agent Systems

Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Yun Liu, Mark Malhotra, Paul Pu Liang, Hae Won Park, Yuzhe Yang, Xuhai Xu, Yilun Du, Shwetak Patel, Tim Althoff, Daniel McDuff, Xin Liu

Google Research, Google DeepMind, MIT

arXiv 2512.08296 (v3, April 2026)

MICHIGAN ENGINEERING
COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY OF MICHIGAN

OVERVIEW

Multi-agent helps on some tasks, hurts on others

THE STUDY

Compared a single agent against four multi-agent designs, on 6 multi-step benchmarks and 9 LLMs. Prompts, tools and total token budget were kept the same.

MAIN RESULT

Multi-agent was 81% better than a single agent on a finance task, but 70% worse on a planning task. Averaged over all tasks, there was no improvement.

KEY FINDINGS

- If one agent already scores above 45%, adding agents makes it worse
- Tasks with many tools lose the most to coordination overhead
- If no agent checks the others' work, errors increase 17×

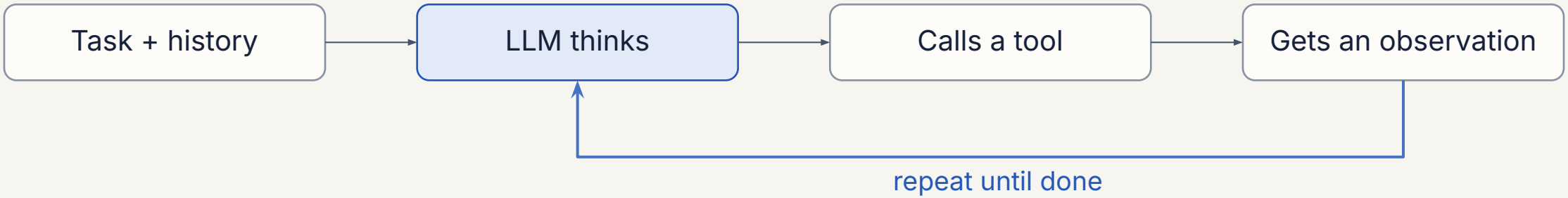
PREDICTION

The authors fit a model that predicts the best design for a task. It picked the right one 87% of the time on setups it had not seen.

Bhargav

BACKGROUND

An agent is an LLM running in a loop



What makes a task agentic:

1. Sustained multi-step interactions with an external environment
2. Iterative information gathering under partial observability
3. Adaptive strategy refinement based on environmental feedback

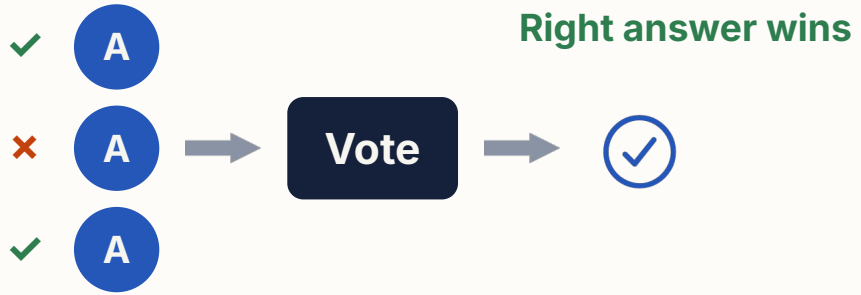
Haripreeth

BACKGROUND

Voting works for one-shot tasks, not multi-step ones

Why use multiple agents? They can split the work, explore options in parallel and check each other.

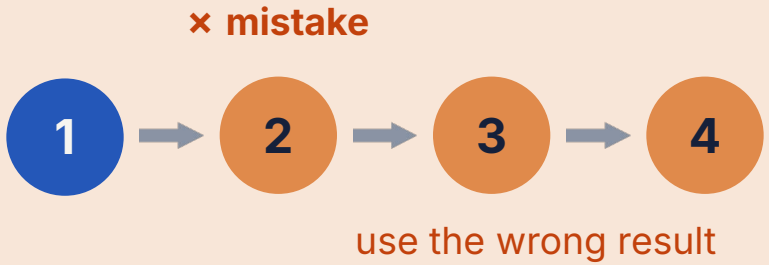
Static task



One wrong answer gets outvoted

Example: 5 agents voting reach 89% on HumanEval (coding)

Agentic task



A mistake in step 2 carries into steps 3 and 4

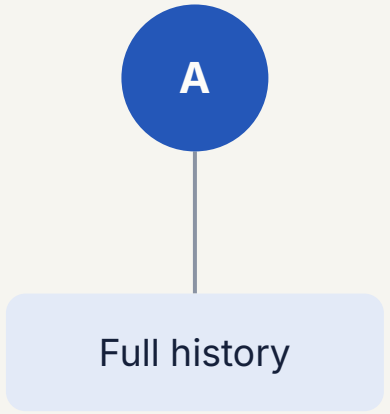
After 10 steps, only 34% of what agents know overlaps

So this paper only tests on multi-step (agentic) tasks.

Bhargav

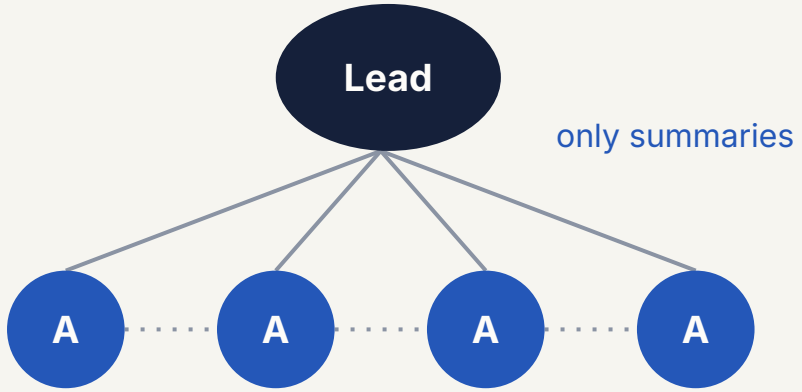
TRADE-OFF

One agent sees everything. A team shares summaries.



One agent: one complete history

About 7 turns per task



Team: a lead and workers, each with its own history

26–44 turns per task when agents coordinate

- Each worker passes on a summary, not its full history, so some details never reach the others.
- Agents also spend turns writing and reading messages instead of doing the task.
- Lost details plus extra turns: that is the **coordination tax**.

Haripreeth

THE PROBLEM

The evidence points both ways

More agents helped

- “More Agents Is All You Need” (2024): more agents with majority voting kept improving accuracy
- Multi-agent debate (Du et al.): agents arguing with each other improved answers
- MetaGPT: a lead agent assigning roles reduced errors

More agents caused problems

- Cognition (2025), “Don’t Build Multi-Agents”: teams were fragile in practice
- Gao et al. (2025): a single strong model often matches a team
- Cemri et al. (2025): found 14 common ways multi-agent systems fail
- Anthropic: agent systems use about 15× more tokens

Teams add communication overhead. This paper measures when the benefit is worth that cost.

Bhargav

WHY IT IS HARD

Earlier comparisons changed everything at once

Earlier studies

One agent		Team
Prompt A	≠	Prompt B
Tools A	≠	Tools B
Budget A	≠	Budget B
1 agent	≠	Team of 3

Team scores higher.
Was it the prompt, the tools, the budget or the design?
You can't tell.

They also reported only final accuracy, so the costs stayed hidden.

This paper

One agent		Team
Same prompt	=	Same prompt
Same tools	=	Same tools
Same budget	=	Same budget
1 agent	≠	**Team of 3**

Team scores higher or lower.
Only the design changed, so the design caused it.

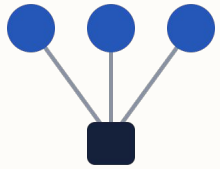
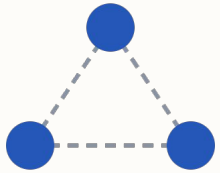
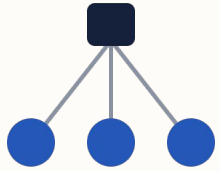
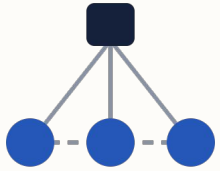
It also records turns, overhead and how errors spread.

Haripreeth

BACKGROUND

Four multi-agent designs, based on two choices

Orchestrator: a lead agent that splits the task, assigns the parts and checks the results.

	Agents don’t talk to each other	Agents talk to each other
No orchestrator	 **Independent** Work alone; answers are merged, no checking	 **Decentralized** Discuss, then vote on an answer
Orchestrator	 **Centralized** Assigns parts, checks results	 **Hybrid** Centralized + agents talk

Bhargav

METHODOLOGY

Six agentic benchmarks

Benchmark	Task	Evaluation Design	Tools	Tasks
BrowseComp-Plus (2025)	Web Browsing / Information Retrieval	Multi-website Information Location	3	100
Finance-Agent (2025)	Finance	Entry-level Analyst Task Performance	5	50
Plancraft (2024)	Agent Planning	Minecraft Environment Planning	4	100
WorkBench (2024)	Planning / Tool Selection	Common business activities	16	100
SWE-bench Verified (2024)	Software Engineering	GitHub Issue Resolution	7	20
Terminal-Bench (2025)	CLI Task Execution	System Admin / Security / ML Tasks	2	20

Table 1 from the paper. The shaded Tools and Tasks columns are added from the paper's setup description.

6 benchmarks × 9 models × 5 designs = 260 runs, all with the same budget

Haripreeth

METHODOLOGY

Nine models, from weak to strong

Model	Index	AA-LCR	HLE	MMLU-Pro	GPQA Diamond	AIME 25	LiveCode	SciCode	IFBench
GPT-5.2	75	73	31	87	90	99	89	52	75
GPT-5	**71**	76	27	87	85	94	85	43	73
GPT-5 mini	**68**	68	20	84	91	84	84	39	75
GPT-5 nano	**59**	42	8	78	84	79	79	37	68
Gemini-3.0 Pro	75	71	37	90	91	96	92	56	70
Gemini-3.0 Flash	75	66	35	89	90	97	91	51	78
Gemini-2.5 Pro	**65**	66	21	86	84	88	80	43	49
Gemini-2.5 Flash	**58**	57	13	84	79	78	63	41	52
Gemini-2.0 Flash	**47**	45*	8*	77	68*	73	39*	35*	30*
Claude Sonnet 4.5	**55**	66	7	88	83	37	71	43	43
Claude Sonnet 4	**47**	62*	5*	87	75	21	56*	38*	35*
Claude 3.7 Sonnet	**42**	58*	2*	81	67	12	57	32*	30*

*Table 6 from the paper. * Estimated or averaged from reported range (the paper's own note). Greyed rows: newer models used only in a later check.*

Haripreeth

METHODOLOGY

How each design actually runs

Design	Who talks to whom	How the final answer is made	Set Up in the paper
One agent	Nobody, it works alone	Its own loop	Up to 10 steps
Independent	Nobody, agents work in parallel	Answers are joined, nobody checks	3 agents
Decentralized	Every agent with every other	Agents debate, then vote	3 agents, 3 debate rounds
Centralized	Each agent with the orchestrator only	Orchestrator splits the work and checks it	1 orchestrator + 3 agents, up to 5 rounds
Hybrid	Orchestrator plus some peer chat	Orchestrator checks, peers also share	Centralized setup + peer rounds

Same total budget for every design, about 4,800 tokens per task.
A team splits it, so each member gets fewer steps than a lone agent, and part of it goes to messages.

METHODOLOGY

What they measure and what they compare

Measured in every run

- Success rate on the task
- Turns taken per task
- Overhead: extra turns compared with one agent
- Message density: messages per turn
- Redundancy: how much agents' work overlaps
- Efficiency: success per unit of turns
- Error amplification: how far errors spread

Comparisons the paper makes

- Each team design against one agent, on every benchmark
- Centralized vs Decentralized vs Hybrid vs Independent
- Weak vs strong models in each family
- Few tools vs many tools
- Team size from 1 to 9 agents
- Mixing strong and weak models in one team
- Cost against performance for each vendor

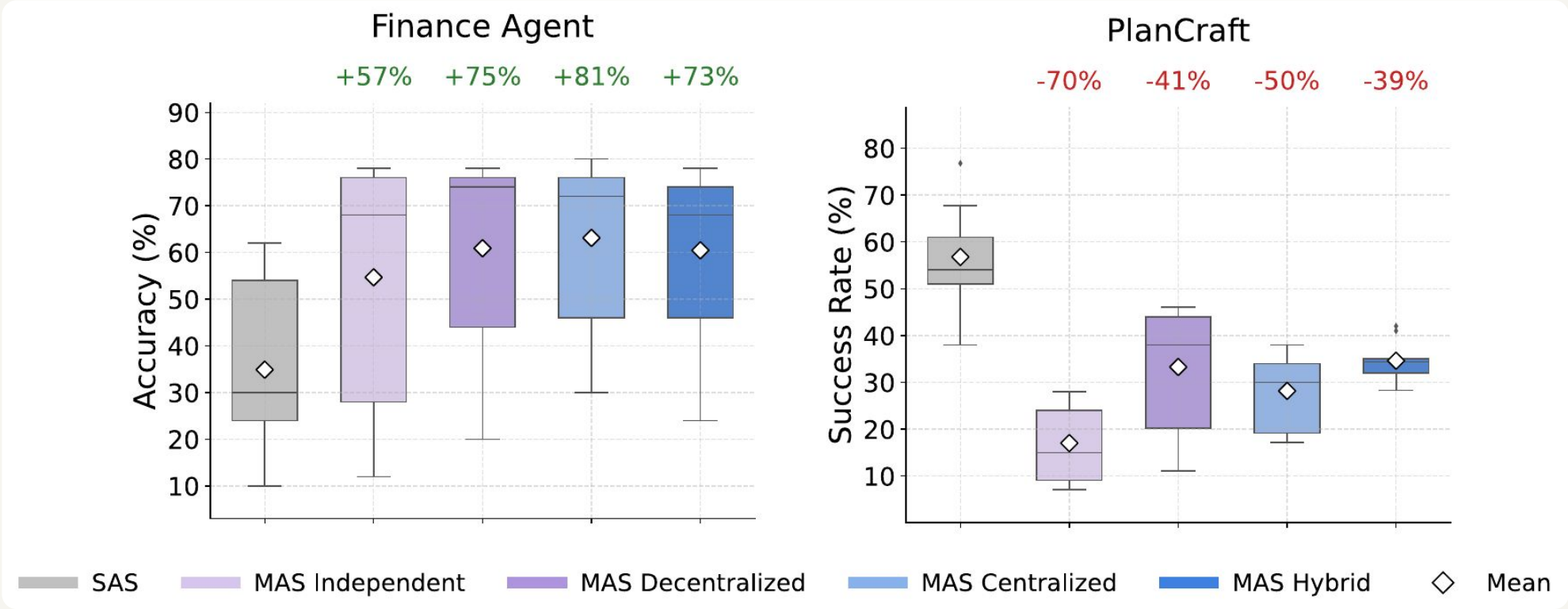
Next: what these comparisons found.

Haripreeth

RESULTS

Same teams: +81% on Finance, −70% on PlanCraft

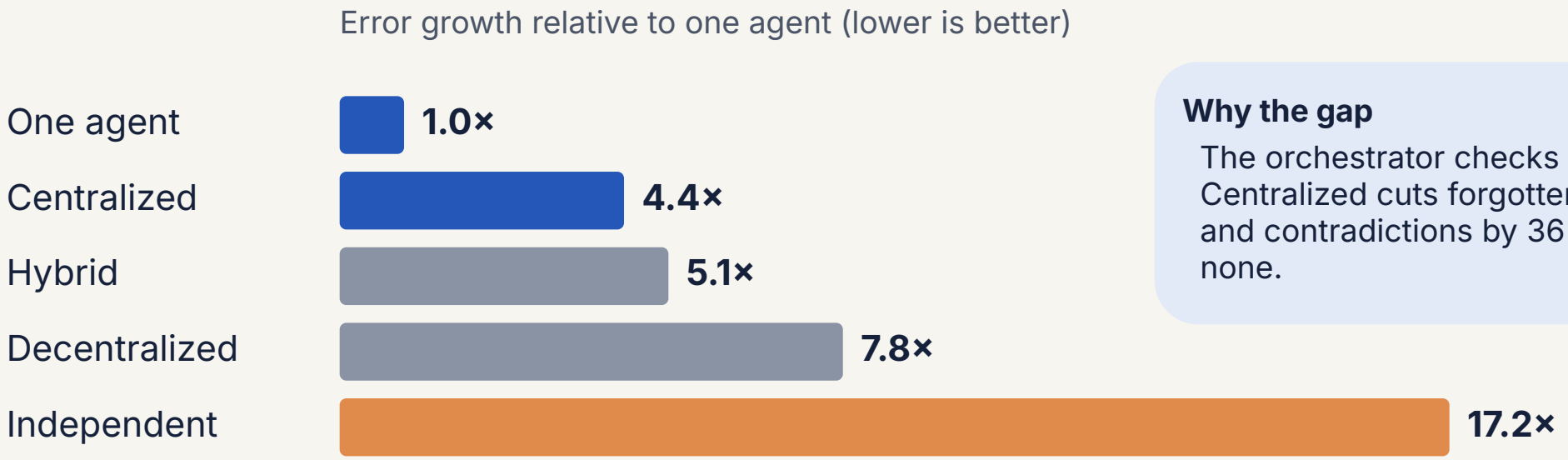
Y-axis: accuracy (higher is better). Gray: single agent. %: change vs. single agent.



Bhargav

RESULTS

Without a checker, errors spread 17×; an orchestrator holds it to 4.4×



Why the gap

The orchestrator checks work before merging it. Centralized cuts forgotten-detail errors by 67% and contradictions by 36%. Independent fixes none.

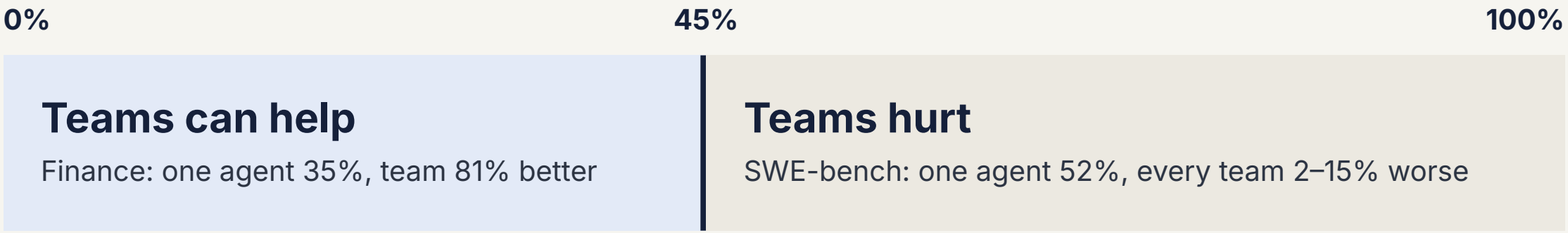
An orchestrator that reviews outputs **limits how far mistakes spread**.

Measured on reasoning traces. On final answers, a team's error rate is only 1.1–1.3× a single agent's.

Haripreeth

RESULTS

Once one agent scores above 45%, more agents hurt

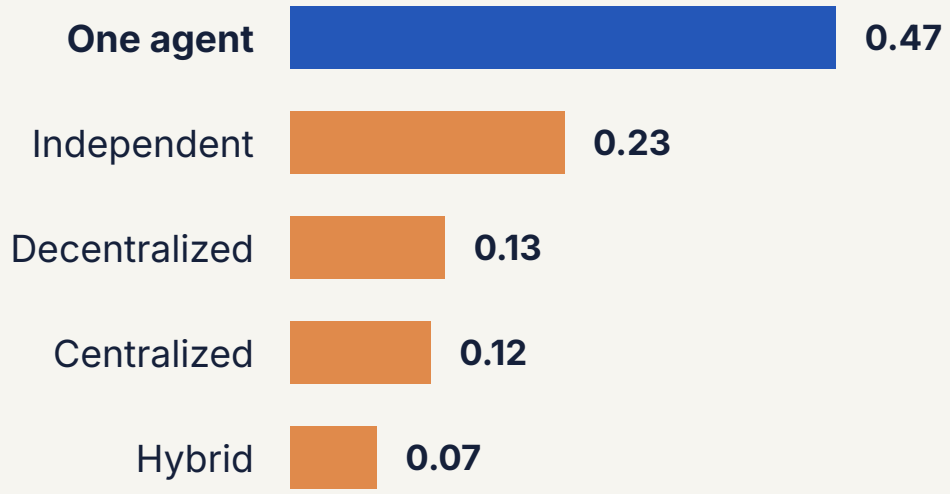


The paper calls this capability saturation. It is their most consistent result.

RESULTS

Tool-heavy tasks pay the biggest coordination tax

Success per turn, by design



Why tools make it worse

The same budget is split across the team

↓

Each agent has less room to plan its tool calls

↓

16 tools (WorkBench): the cost adds up
4 tools or fewer: almost no penalty

The more tools a task needs, the worse a team does compared with one agent.

Haripreeth

RESULTS

Good at picking designs, weak at exact scores

37%

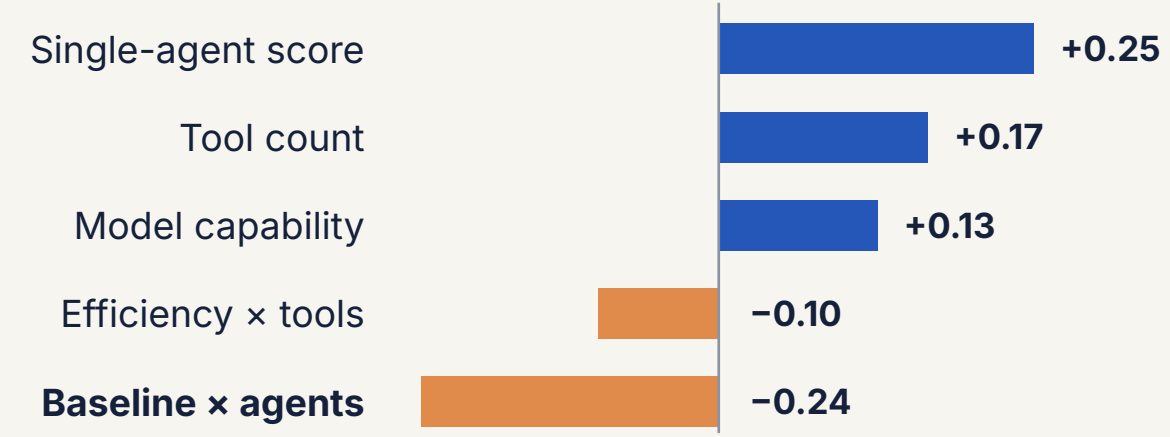
of the variation it explains on runs it has not seen

87%

of the time it picks the best design

What moves performance

Right helps, left hurts



Biggest negative: when one agent is already good, adding agents hurts.

The formula, simplified

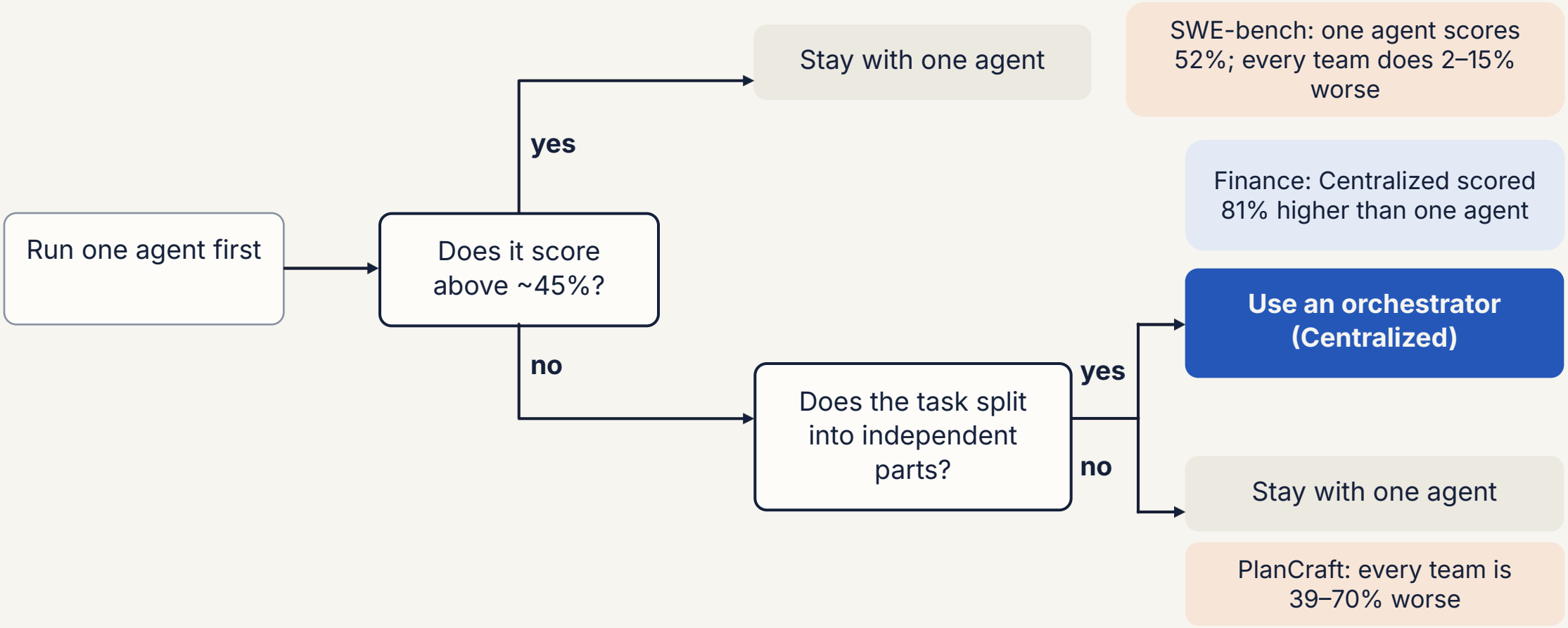
$$P = \beta_0 + \beta_1 \cdot I + \beta_3 \cdot \log(1+T) + \beta_{10} \cdot P_{sa} + \mathbf{\beta_{17} \cdot (P_{sa} \times \log(1+n_a))} + \ldots + \varepsilon$$

I model capability · T tools · n_a agents · P_{sa} single-agent score · … 15 more terms, mostly coordination costs

Haripreeth

RESULTS → PRACTICE

Which design should you use?



Bhargav

OUR TAKE

A strong study, a weaker predictor

What holds up

- Fair test: same prompts, tools and budget
- The ~45% rule survives every check
- Task structure matters more than team size
- Patterns held on 3 newer models

Why the predictor is weak

- Explains only 37% of unseen runs
- Coordination metrics add little: 0.360 → 0.373
- Only 3 of 19 terms survive their own checks
- On new models it picked Hybrid; Hybrid lost

Haripreeth

PAPER 2 OF 2

FlashAgents: Accelerating Multi-Agent LLM Systems via Streaming Prefill Overlap

Taosong Fang, Zhen Zheng, Zhengzhao Ma, Yaojie Lu, Hongyu Lin, Xianpei Han, Le Sun
Institute of Software, CAS · UCAS · Microsoft
MLSys 2026

MICHIGAN ENGINEERING
COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY OF MICHIGAN

B idles until A's last token arrives

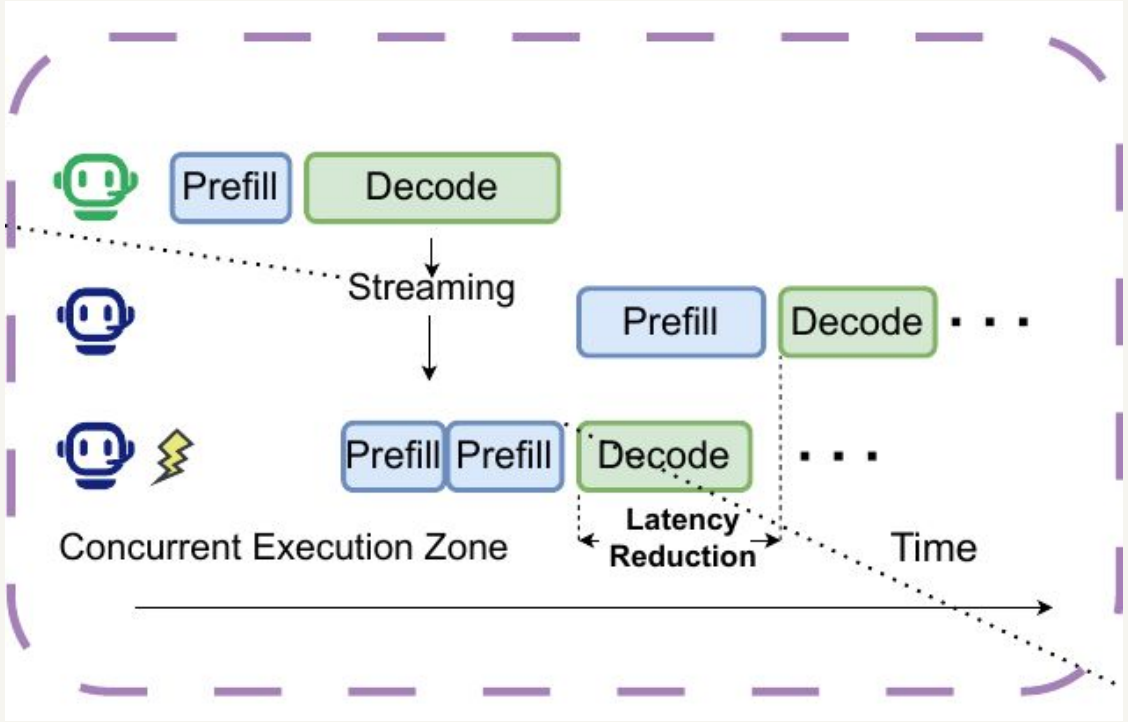


Fig. 4 (middle): B's prefill chunks run inside A's decode window.

Up to 40% lower end-to-end latency on real workflows; up to 3.5× in two-agent benchmarks.

Inter-agent dependency

A's output is B's input. Serving systems optimize one request or batch independent ones; none optimize the gap between dependent calls.

Standard serving is strictly sequential

Each agent call is a separate request, so B starts prefill only after A's full generation. The idle gap repeats at every handoff and turn.

FlashAgents overlaps the two phases

A streams tokens to B as it decodes; B prefills them causally in chunks and prefetches weights and KV before A finishes.

Most of B's prompt exists before A speaks

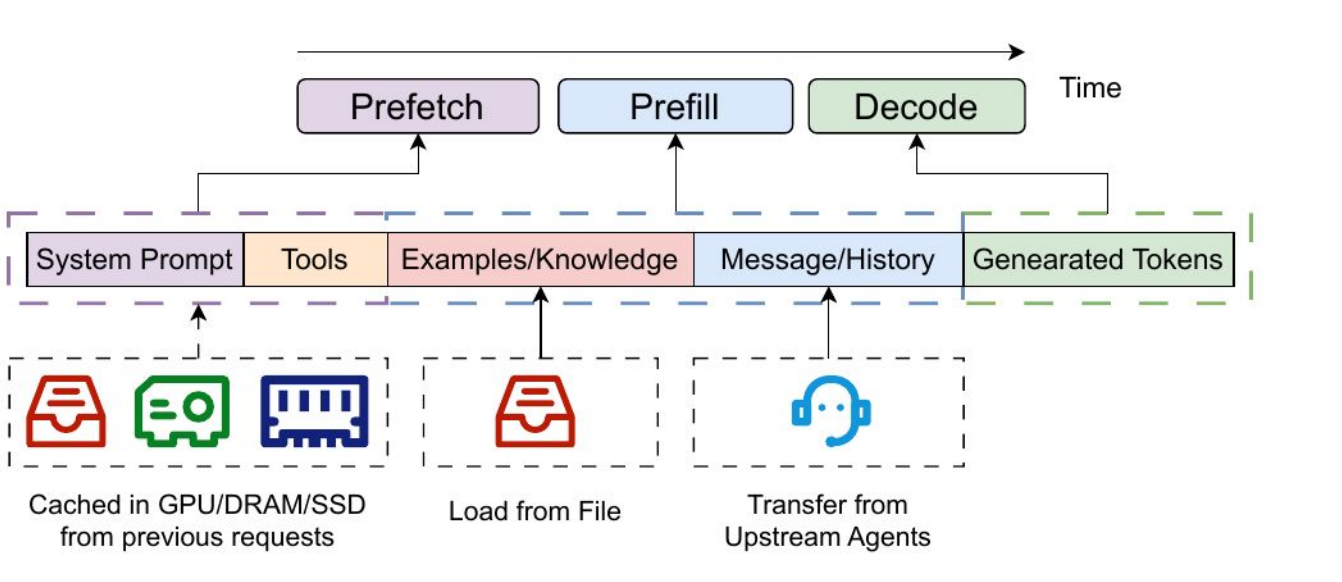


Fig. 2: context composition and execution timeline.

System prompt + tools
Prefix-cached in GPU/DRAM/SSD from earlier requests → prefetch

Examples / knowledge
Loaded from file → prefill any time

Message / history
Transferred from upstream agents → waits on A

Generated tokens
B's own decode output

Only the Message/History slice depends on A. Prefetch and most of prefill can start early; streaming moves the rest off the critical path.

Every MAS pattern has sequential handoffs

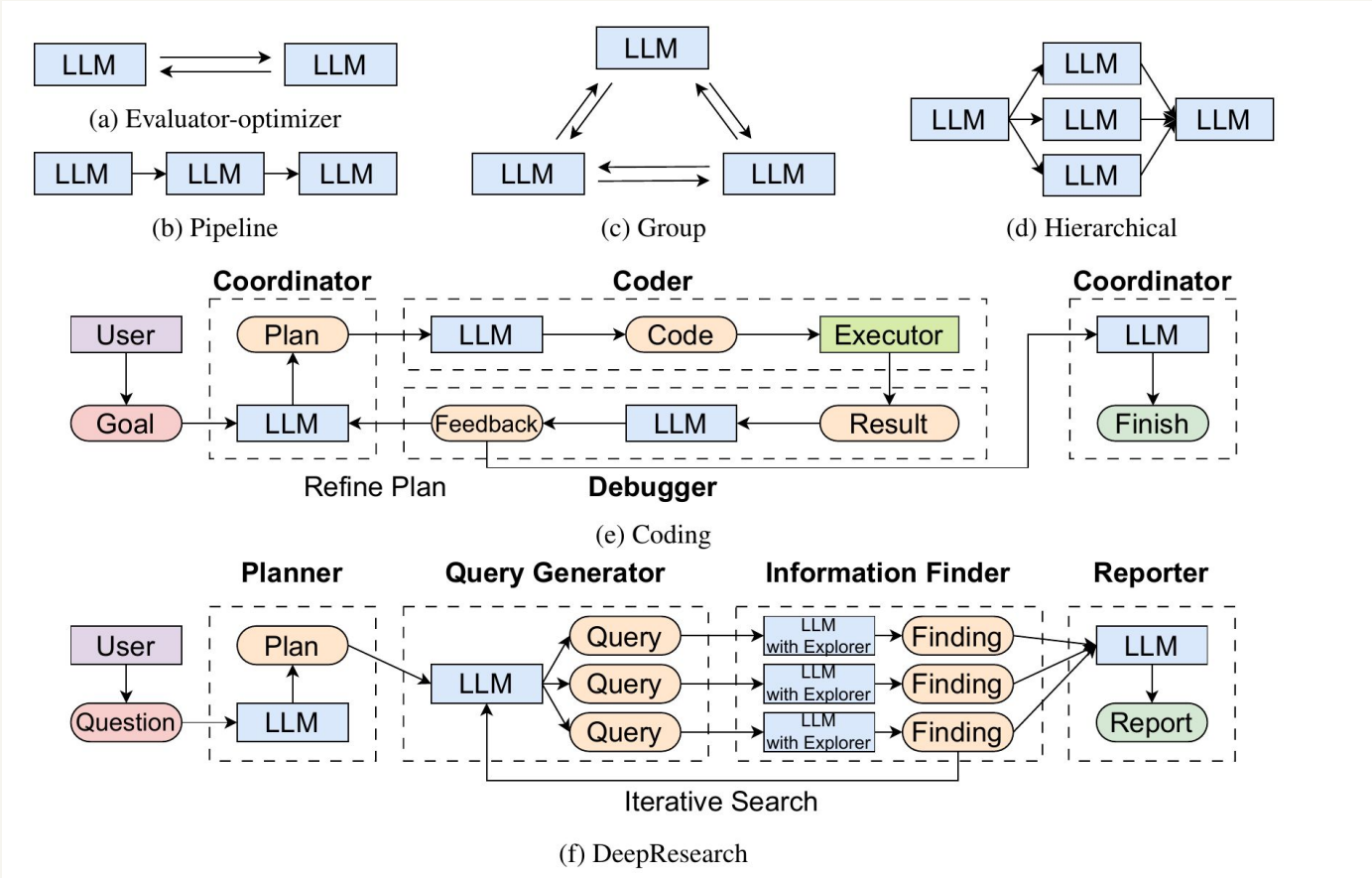


Fig. 3: (a)–(d) workflow templates; (e) coding and (f) DeepResearch dynamic workflows.

Templates (a)–(d)

Evaluator-optimizer, pipeline, group, hierarchical: fixed code paths.

Dynamic (e)–(f)

Coding and DeepResearch: the execution graph appears at runtime.

Heterogeneous models

Small upstream model (8B) decodes fast; large downstream model (32B) has heavy prefill, so the overlap window widens.

Every edge is a decode → prefill dependency.

Prefill is only ~3× faster than decode

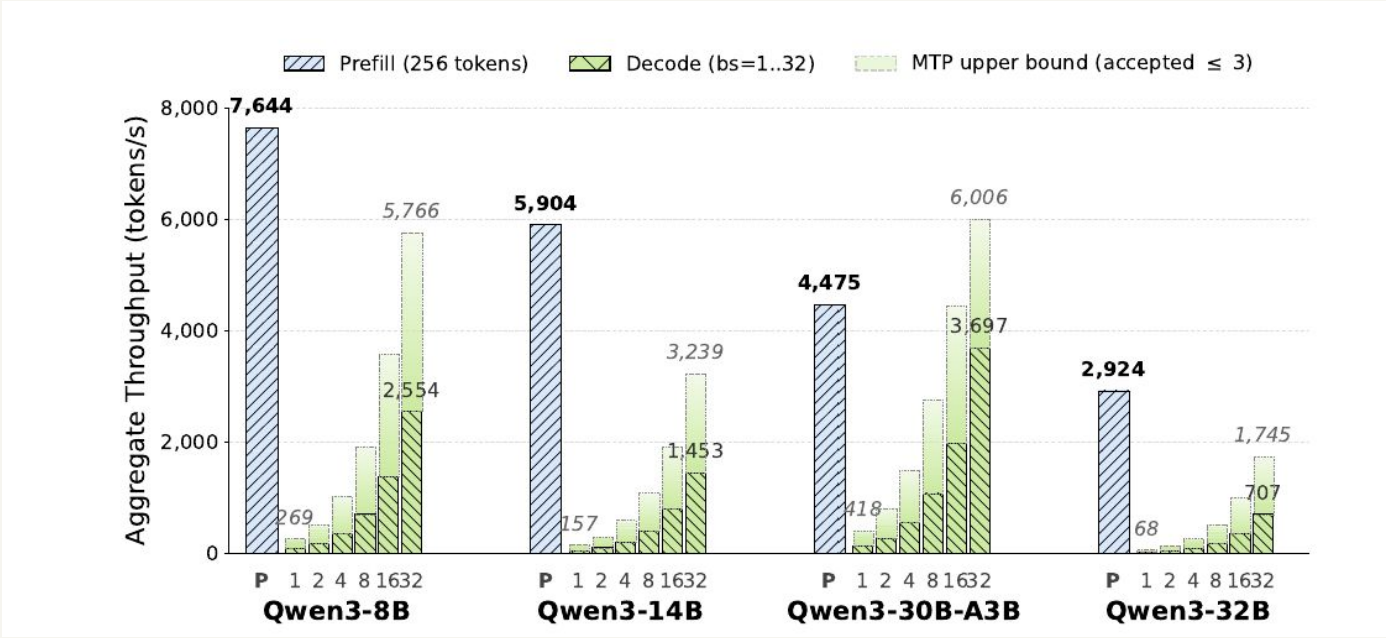


Fig. 1: prefill vs. decode aggregate throughput, A800-80GB (P = prefill; 1–32 = decode batch).

Prefill: compute-bound
Processes the whole prompt in parallel, builds the KV cache, sets TTFT.

Decode: bandwidth-bound
One token per step, reading the KV cache; sets TPOT.

7,644 tok/s
Qwen3-8B prefill (256 tokens)

2,554 tok/s
Qwen3-8B decode, batch 32 (5,766 with MTP)

Same order of magnitude, so prefill hidden under decode removes real work from the critical path.

Gap in prefix caching. SGLang and vLLM insert into their radix tree only after prefill finishes, so requests in the same turn cannot share a prefix.

B prefills A's tokens chunk by chunk

Algorithm 1 Core Inter-agent streaming and incremental prefill Mechanism (AGENT A → AGENT B)

Input: Prompt $P_{\mathcal{A}}$ for AGENT A; engines $E_{\mathcal{A}}, E_{\mathcal{B}}$ with KV cache $KV_{\mathcal{B}}$

1: $Buf_{\mathcal{B}} \leftarrow \emptyset$; $KV_{\mathcal{B}} \leftarrow \emptyset$
2: $E_{\mathcal{A}}$.StartDecode($P_{\mathcal{A}}$)
3: **while** $E_{\mathcal{A}}$ is decoding **do**
4: $tok \leftarrow E_{\mathcal{A}}$.NextToken()
5: $Buf_{\mathcal{B}}$.Append(tok)
6: **if** $|Buf_{\mathcal{B}}| \geq$ threshold **or** tok is final **then**
7: $E_{\mathcal{B}}$.Prefill($Buf_{\mathcal{B}}, KV_{\mathcal{B}}$) ▷ updates prefix cache
8: $Buf_{\mathcal{B}} \leftarrow \emptyset$
9: **end if**
10: **end while**
11: **return** $E_{\mathcal{B}}$.Decode($KV_{\mathcal{B}}$)

Algorithm 1: core mechanism (Agent A → Agent B).

1. **Stream**
   E_A appends each generated token to buffer Buf_B (lines 2–5).

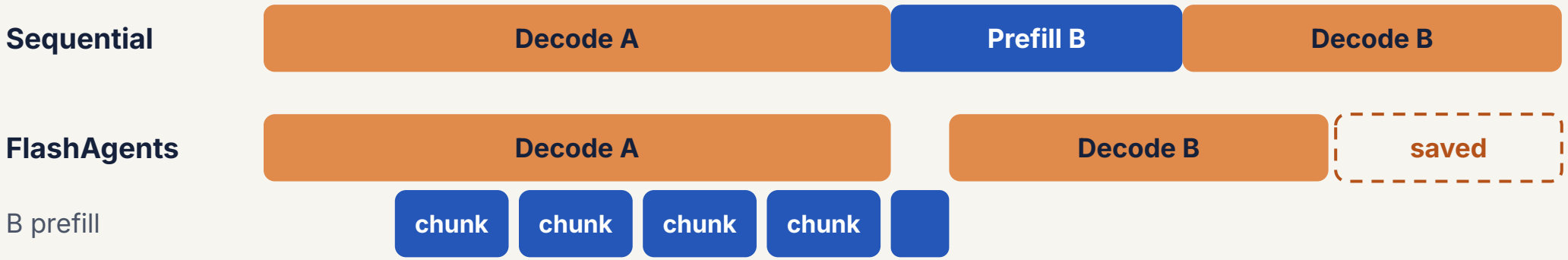
2. **Trigger**
   When |Buf_B| ≥ threshold or the token is final, E_B prefills the chunk (lines 6–8) and prefetches weights and cached KV.

3. **Causal KV update**
   Chunk i attends to every earlier key: softmax(Q_i·K_:iᵀ / √d)·V_:i (Eq. 1). Query length n_i stays small; keys grow.

4. **Drain and decode**
   After A's final token, leftover tokens are prefilled and B decodes from the complete KV_B (line 11).

LATENCY MODEL · §3.1

The saving is capped by the shorter phase



Sequential (Eq. 2)

L_seq = T(dec A) + T(pre B) + T(dec B)

Streaming (Eq. 3–4)

L = T(dec A) + T(pre B) − T(overlap) + T(dec B)

T(overlap) ≤ min(T(dec A), T(pre B))

N concurrent pairs (Eq. 5)

L_N uses T′(pre B) ≤ T(pre B): co-batched chunks and intra-turn prefix reuse shrink prefill.

7B model: 1.05× at concurrency 1 → 3.5× at higher concurrency. The gain comes from GPU utilization and prefix reuse, not more overlap per pipeline.

Each shared prefix is prefilled once

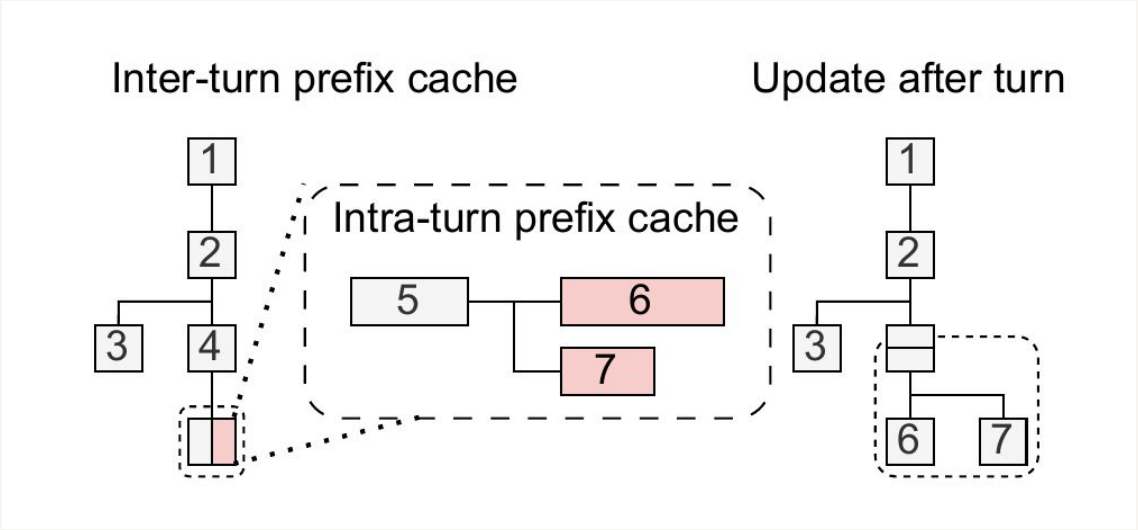


Fig. 5: red nodes are new this turn; each is prefilled once and reused.

1. **Build:** Insert every buffer U_r into a temporary radix tree T; shared spans collapse into one node.
2. **Compute:** Prefill only new nodes, shallow to deep, each extending its parent's KV.
3. **Propagate:** Each request gathers KV deltas along its root-to-leaf path.

Algorithm 2 Prefix-Aware Aggregation for Incremental Prefill

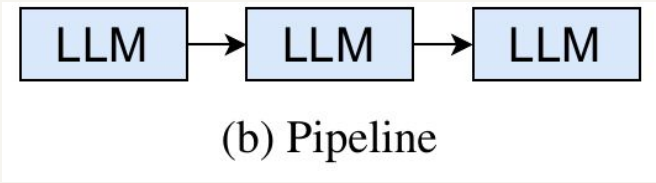
Input: Active requests $\mathcal{R}$ with unbuffered tokens U_r; radix tree $\mathcal{T}$

for request $r \in \mathcal{R}$ with $U_r \neq \emptyset$ **do**
  $\text{Insert}(\mathcal{T}, U_r, r)$ ▷ create nodes for unique prefixes
end for
$\mathcal{N} \leftarrow$ newly created nodes in $\mathcal{T}$, ordered by depth
for node $n \in \mathcal{N}$ **do** ▷ from shallow to deep
  $KV_n \leftarrow E.\text{Prefill}(\text{tokens}(n), KV_{\text{parent}(n)})$
end for
for request $r \in \mathcal{R}$ **do**
  for node n on path from root to r **do**
    $KV_r \leftarrow KV_r \cup KV_n$
  end for
end for

Cost: building T is O(Σ|U_r|); memory grows with unique prefix extensions; causal order holds along each path.

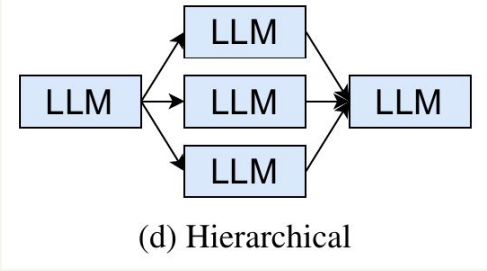
Fan-in waits for position-stable segments

Linear chain



A streams to B, B to C: every link pipelines. The gain accumulates but the slowest link bounds it.

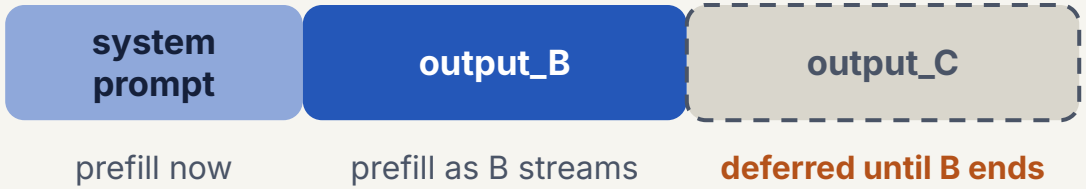
Fan-out and fan-in



From Fig. 3 (b) and (d)

Fan-out: A streams the same tokens to B and C in parallel. Fan-in: D needs both, which raises the position problem.

Fan-in {B, C} → D: D's prompt



RoPE encodes position before attention, so a token's KV is only valid at its final position. output_C starts at |system| + |output_B|, which is unknown until B finishes.

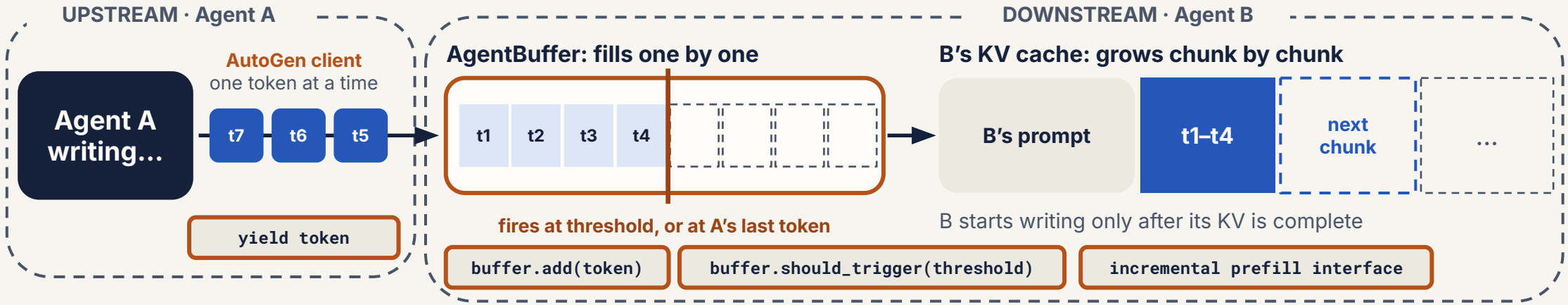
Rule: incrementally prefill only position-stable prefixes and defer the rest. It stays correct and still overlaps everything before the first unknown length.

IMPLEMENTATION

How FlashAgents is built on SGLang

Idea 1: Streaming + incremental prefill

Bold outline = new code FlashAgents adds to SGLang



Idea 2: Prefix sharing: B requests that start with the same text



EVALUATION

How it's tested: microbenchmark + real workflows

MICROBENCHMARK

Does B's wait shrink?

- **Workload:** A → B pipeline
- **Varies:**
  - A's speed (TPS)
  - Shared prefix length
  - Tokens streamed A → B
- **Concurrent requests:** 1, 2, 4 or 8
- **Models:** Qwen2.5-7B, Qwen3-235B
- **Metric:** T = A's first token → B's first output token
- **Baseline:** B starts after A ends (plain SGLang)

REAL WORKFLOWS (END-TO-END)

Does it help real apps?

- **Workloads:** 3 heterogeneous agentic workflows (different models per agent)
  - **Chain:** 4-agent travel planner
  - **MapReduce:** 3 reviewers → 1 meta-reviewer
  - **PPTAgent:** research paper → 12 slides
- **Concurrent requests:** 1 or 8
- **Metric:** end-to-end latency vs. baseline
- **Baseline:** agents run one by one (AutoGen default)

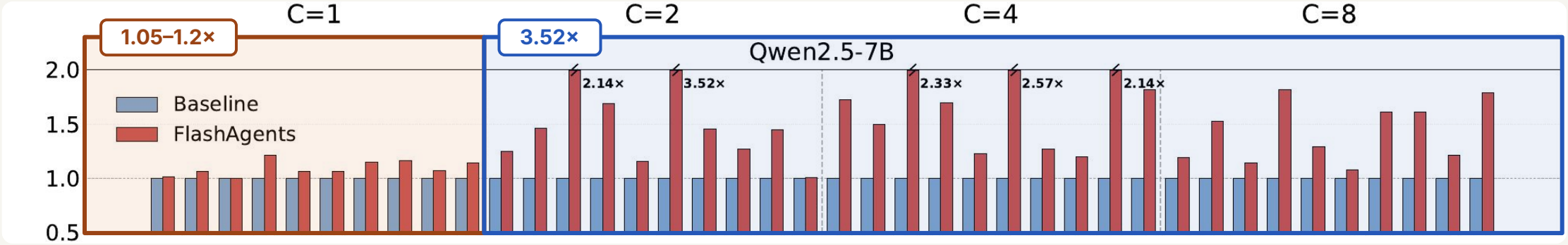
OTHER STUDIES

- **Ablation →** isolates which part helps (streaming vs. prefix cache)
- **Overhead →** cost of small prefill chunks
- **Orthogonality →** still helps on top of SJF scheduling

RESULTS

Microbenchmark (Qwen2.5-7B): up to 3.52× faster

Y-axis: times faster than baseline (1.0 = same speed, 2.0 = 2× faster) **X-axis:** workloads, grouped by C = A→B pipelines on one GPU



1.05–1.2×

1 pipeline (C = 1)
B's prefill is short: little to overlap.

3.52×

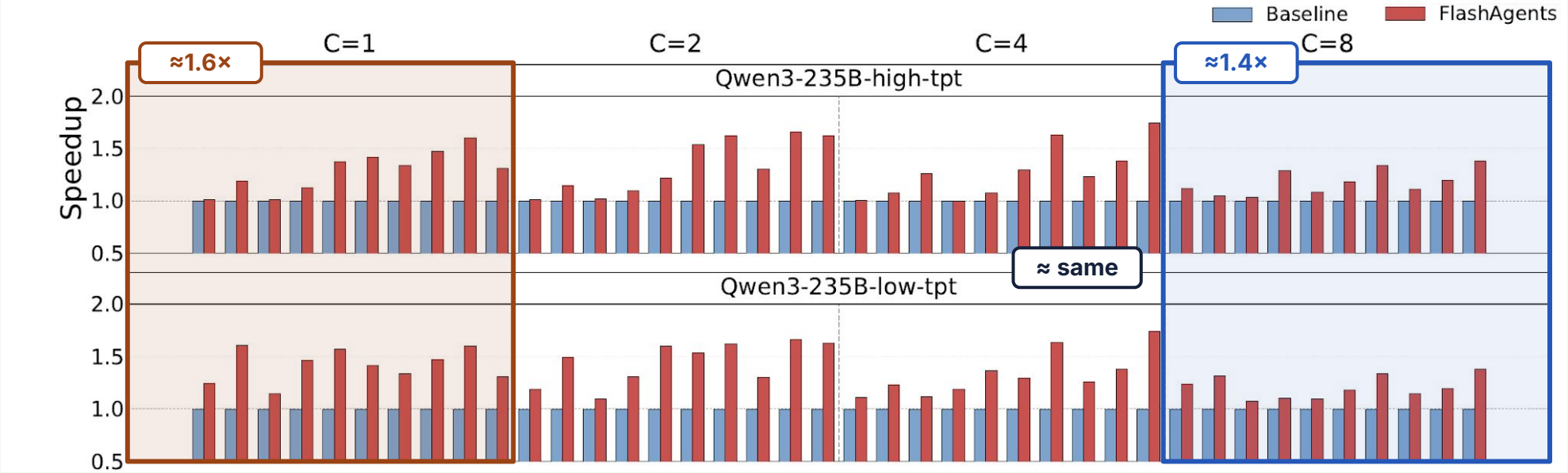
2, 4 or 8 pipelines at once (C ≥ 2)
Several Bs prefill while A writes, and their shared prompt is computed once.

Never slower than the baseline in any setting tested (7B and 235B).

RESULTS

Qwen3-235B: gains level off as C grows

Y-axis: times faster than baseline (1.0 = same speed) **X-axis:** workloads, grouped by C = A→B pipelines (8× A100 GPUs)



≈1.6× — **C = 1: more than 7B**
Big model, slow prefill: more of it overlaps.

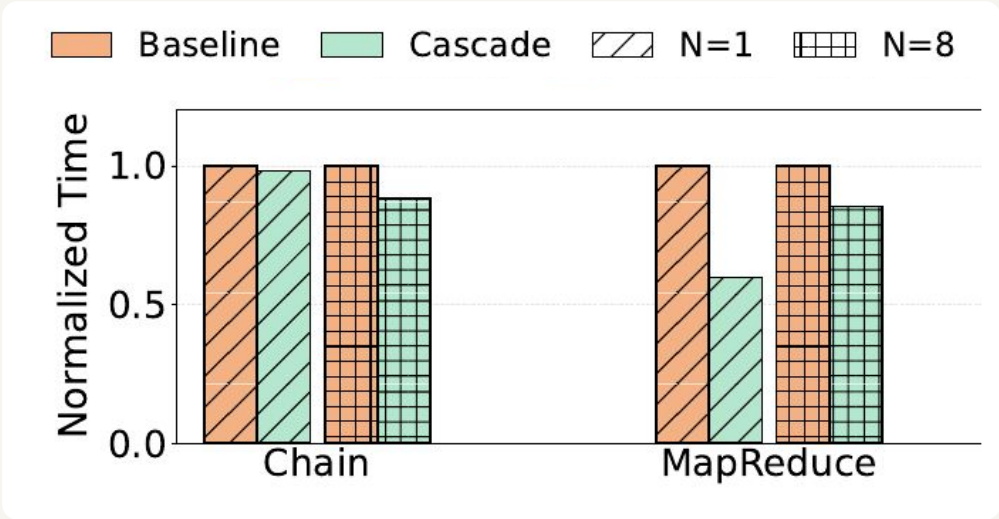
≈1.4× — **C = 8: gain levels off**
Down from ≈1.76× at C = 4: all Bs' prefill outlasts A's decode.

≈ same — **Fast A vs. slow A**
Slow A (bottom) a bit higher: A decodes longer, more to overlap.

RESULTS

Real workflows: up to 40% lower latency

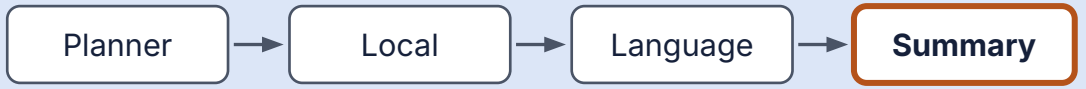
Y-axis: time vs. baseline, lower is better **"Cascade"** = FlashAgents **N =** requests at once (1 or 8)



Bold outline (right) = agent pre-warmed by FlashAgents

Chain · travel planning

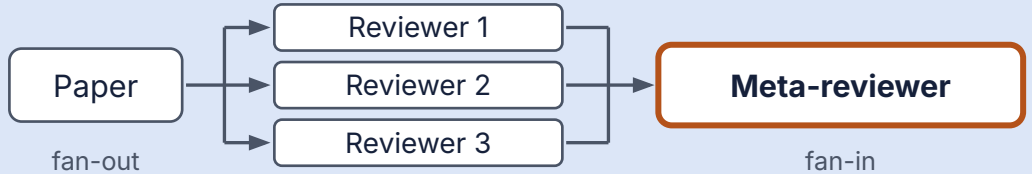
N=1 −2% N=8 −12%



- **Pre-warmed:** 98% of Summary's prompt, during Planner.
- **Small gain:** prefill is a tiny slice of 4 agents in a row.
- **N=8:** shared guide computed once → −12%.

MapReduce · paper review

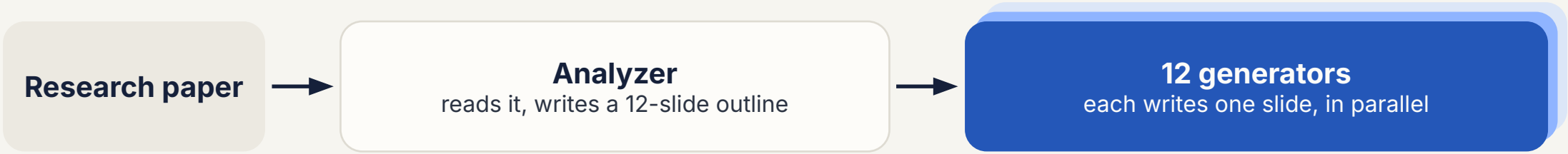
N=1 **−40%** N=8 −15%



- **Parallel:** reviewers run together (baseline: one by one).
- **Pre-warmed:** meta-reviewer's shared paper prefix.
- **N=8:** busy shared 32B GPU → −15%.

RESULTS · PPTAGENT

PPTAgent: a paper becomes 12 slides, 24% faster



What each of the 12 generators reads



End-to-end time



RESULTS · ABLATION

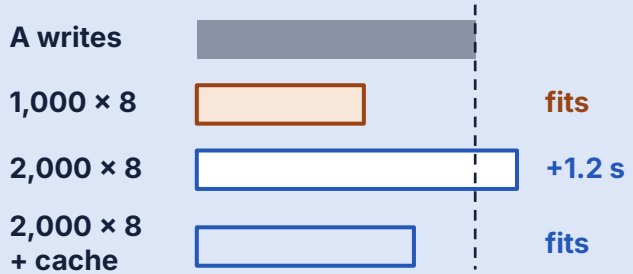
Prefix cache helps only with long, shared prompts

Latency in seconds, lower is better · Qwen3-32B · **pipelines** = A→B pairs running at once

Shared prompt	Pipelines	Baseline	Streaming only	+ Prefix cache	Speedup
1,000	1	4.46	4.16	4.11	**1.09×**
1,000	8	6.93	**4.11**	**4.11**	**1.69×**
2,000	1	4.77	4.15	4.11	**1.16×**
2,000	8	10.10	**5.32**	**4.20**	**2.40×**

Why only 2,000 × 8 gains

B can't finish before A, so ≈4.1 s is the floor.



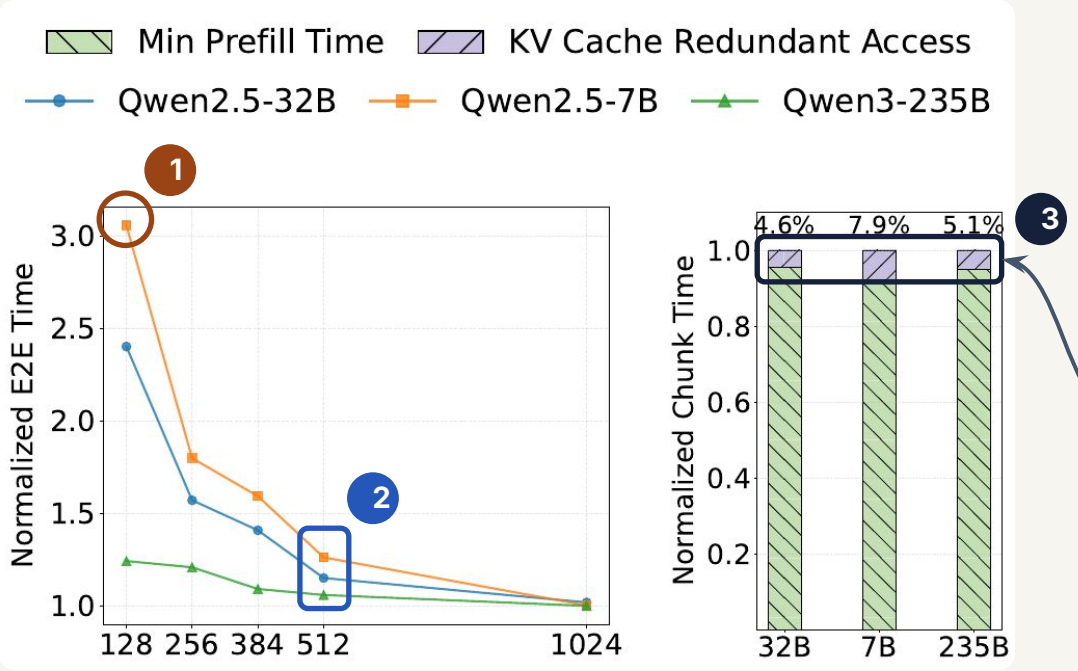
Sketch of B's prefill work vs. A's writing (our reading)

Streaming gives most of the gain; the cache adds more only at 8 pipelines × 2,000 tokens.

RESULTS · OVERHEAD

The cost: smaller chunks make B's prefill slower

X-axis: chunk size (tokens) **Y-axis:** B's prefill time vs. 1,024-token chunks (1.0 = no extra cost)



1 **3×** **slower with 128-token chunks**
7B model; bigger models pay less

2 **≤1.3×** **slower with 512-token chunks**
all models

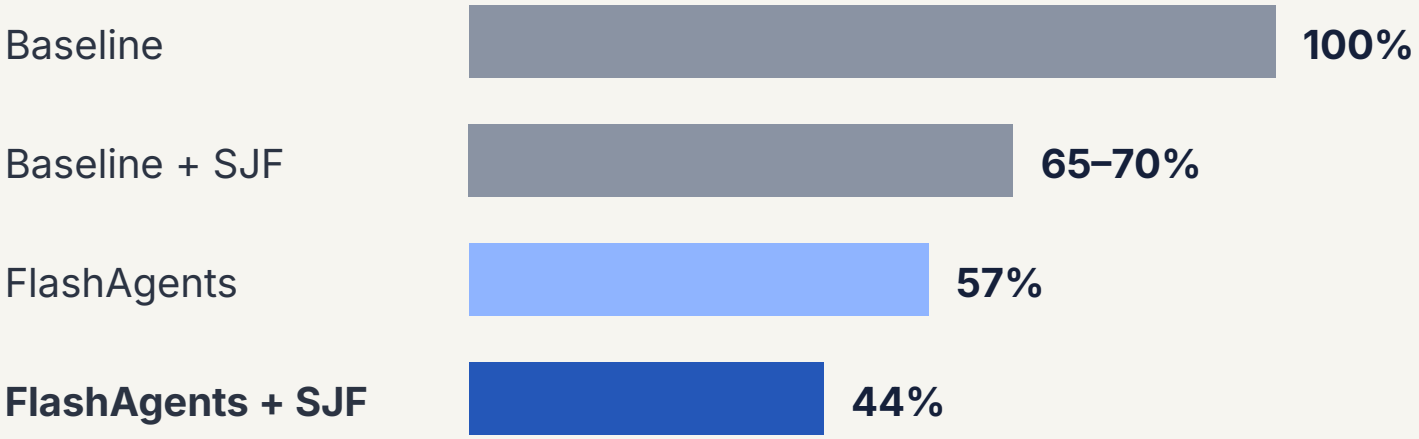
3 **Last chunk vs. first:** re-reading 896 earlier tokens adds only 5–8%.

Chunk size is the knob.
Smaller → B starts sooner, but costs more.

RESULTS · ORTHOGONALITY

FlashAgents stacks with smarter scheduling

Bars: latency as % of baseline, lower is better **SJF:** shortest job first **Setting:** 1 request at a time



Why they add up

FlashAgents: works within a workflow.

SJF: short requests run first.

Different levels → gains add up.

Combined: 56% lower latency at 1 request, 37% at 10 requests.

OUR TAKE

Strengths and weaknesses

Strengths

Targets two separate delays
Streaming: B starts early. Cache: shared prefix prefilled once.

Shares prefill across concurrent requests
Requests arriving together share one prefill; SGLang can’t.

Shown in real applications
Built on SGLang + AutoGen; tested on 3 real workflows.

Weaknesses

MapReduce’s 40% isn’t all FlashAgents
Reviewers run in parallel; the baseline runs them in turn.

The 1,000-token result is unexplained
Cache gives no gain; GPU prefill time would show why.

Starting early costs GPU time
Small chunks: up to 3× prefill time. Worth it on a busy GPU?

CONCLUSION

Coordination has a price: three layers to manage it

PAPER 1

Should you use a team?

Only when the task splits into parts. Otherwise coordination costs more than it gives.

OPTIONAL READING · ORLA

Run the workflow well

Right model per stage, smart order, and KV cache kept across stages.

PAPER 2 · FLASHAGENTS

Make each handoff cheaper

B starts before A finishes; the shared prompt is prefilled once.

The gap: Orla waits for each stage to finish. FlashAgents removes that wait.

What each one gives us

- **Paper 1:** a rule for when a team helps: match it to the task (−70% to +81%).
- **Orla:** a serving layer for the whole workflow (−38% time, −35% cost).
- **FlashAgents:** faster handoffs with the same output (up to 40% lower latency).